CS311 Operating System

School of Information Technology, Engineering, Mathematics and Physics

Final Examination
Semester 1 2025

F2F

Duration of Exam: 3 hours + 10 minutes

Reading Time: 10 minutes

Writing Time: 3 hours

Total marks: 100

Instructions:

1. *This exam has three Sections:*
    a. *Section A Short Answer 20 Marks*
    b. *Section B Problem Solving and Analysis 60 Marks*
    c. *C Case Study. 20 Marks*

   ALL sections are compulsory.
2. *There are 16 pages in this paper including this cover page.*
3. *Write your answers directly in the space provided.*
4. *Show all necessary working. Partial marks will be awarded for partially correct answers.*
5. *Only non-programmable calculators are allowed.*
6. *This exam is worth 45 % of your overall mark.*
7. *The minimum passing mark is 40/100.*

Student ID: _____

Student Name: _____

Section A: Short Answer Questions 20 Marks

There are 5 short answer questions. All are compulsory. Each question is weighting 4 Marks.

Question 1 4 Marks

A. Explain the role of interrupts, device drivers and the kernel in an operating system.

B. Sketch the diagram to show the abstract view of the components of a computer system.

C. For each of the following scenarios, identify the system call categories being utilized:

i. Transitioning from Running State to Waiting State.

ii. The system records the timestamp of the logged in user.

iii. If a program tries to divide by zero, the system displays an error message.

iv. The operating system gives each application a slice of the processor's time.

v. Sensitive information is scrambled to avoid unauthorized disclosure of information.

Question 2 4 Marks

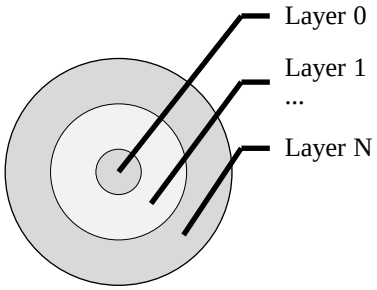
A. Identify at least four primary approaches for interacting with an operating system.

B. Give three reasons that cause a process to move from the running state. Use a diagram to support your answer.

Question 3 **4 Marks**

A. Company X requires a backup solution for critical data stored in Site A's server (Size of data is approximately 800GB). The company had acquired 2 identical SSDs with 1TB capacity each. You are required to do an analysis for selecting the best solution for RAID and determine whether RAID 0, RAID 1 or RAID 5 would be the optimal solution. Use a diagram to support your answer. Include details of redundancy, disk utilization, backup and recovery options.

B. Use the diagram to answer the following questions:



i. Explain the purpose of Layer 0 and the impact it has on Layer N. Would this design be applied to virtual machines.

ii. State one advantage and one disadvantage of using this design when compared to monolithic structure.

Question 4 **4 Marks**

A. Use a while loop program structure to illustrate the elements of a program code during process synchronization.

B. A solution to the critical-section problem must satisfy three requirements. Describe these requirements.

C. Identify and explain the APIs (Application Programming Interfaces) used in UNIX systems, Apple Macintosh, and Microsoft's Windows 11

Question 5 **4 Marks**

A. Kernels are generally supported by multithreaded models. Distinguish between the different multithreaded models used in operating systems. Explain how each model impacts concurrency.

B. Describe the four necessary conditions that characterize deadlocks.

Section B: Problem-Solving and Analysis 60 Marks

There are 6 problem-solving and analytical questions. All are compulsory. Each question is weighting 10 Marks.

Question 1 10 Marks

Given a disk schedule with a queue of 5000 cylinders, numbered 0 – 4999. The hard disk drive is currently serving a request at cylinder 2250 and the previous request was at cylinder 2150. The queue of pending requests, in FIFO order, is 3500, 1300, 2300, 3800, 500, 1200, 2200, 700, 1000, 3650. Evaluate the use of FCFS, SSTF, CSCAN, CLOOK and elevator disk scheduling algorithms and recommend the optimal solution. Use diagrams, calculations and critical analysis to support your answer.

Question 2 **10 Marks**

A. A paging system uses 16-bit address and 4K pages. The page tables of two running processes, Process 1 and Process 2 are shown below. Translate the given **logical addresses** to **physical addresses** by identifying the **page number** and **offset** and using the page table to find the **frame number**. 6 Marks

Process 1

Page Number	Frame Number
0	5
1	4
2	0
3	2

Process 2

Page Number	Frame Number
0	6
1	3
2	2
3	4

Process	Logical Address	Page Number	Offset	Physical Address
1	12000			
2	5000			

B. Consider the page table for a system with **12-bit** virtual and physical addresses and **64-byte** pages. 4 Marks

PAGE TABLE	
Page Number	Frame Number
0	1
1	4
2	-
3	8
4	6
5	2
6	-
7	-
8	9
9	3

The list of free frames is X, Y,Z (that is, X is at the head of the list, Y is second and Z is last). A dash for a page frame indicates that the page is not in memory. Convert the following virtual addresses to their equivalent physical addresses in hexadecimal. All numbers are given in hexadecimal.

i. 23F
ii. 0FF
iii. 10B

Question 3 **10 Marks**

i. Given eight memory partitions of 12KB, 4KB, 20KB, 18KB, 10KB, 5KB, 15KB, and 17KB (in order), how would the first-fit, best-fit, and worst-fit algorithms place the following processes:

- P1 (requires 15KB), P2 (requires 12KB), P3 (requires 10KB)
- P4 (requires 5KB). P5 (requires 16 KB).

Clearly indicate using the memory maps given below how each process is contiguously allocated. Also indicate any fragmentation. (If applicable – Internal/External fragmentation). 4 Marks

First Fit Algorithm

12KB		**4KB**		**20KB**		**18KB**		**10KB**		**5KB**		**15KB**		**7KB**

Worse Fit Algorithm

12KB		**4KB**		**20KB**		**18KB**		**10KB**		**5KB**		**15KB**		**7KB**

Best Fit Algorithm

12KB		**4KB**		**20KB**		**18KB**		**10KB**		**5KB**		**15KB**		**7KB**

ii. Consider the following page reference string assuming demand paging with three frames:

7, 2, 3, 1, 2, 5, 3, 4, 6, 7, 7, 1, 0, 5, 4, 6, 2, 3, 0, 1

i. Compute the page faults using FIFO replacement, LRU replacement and Optimal replacement. 3 Marks

ii. Analyse the results and indicate the algorithm with a better performance and would this be implemented in the real-world scenario. 1 Mark

iii. Assuming there are four frames, draw a Diagram showing the FIFO Replacement and clearly indicate Belady's Anomaly. (If applicable) 2 Marks

Question 4 **10 Marks**

A. Consider the following snapshot of a system in which five resources A, B, C, D, and E are available. The system contains a total of 3 instances of A, 2 instance of resource B, 1 instance of resource C, 2 instances of resource D and 1 instance of resource E.

	Allocation					**Request**					**Available**				
Process	A	B	C	D	E	A	B	C	D	E	A	B	C	D	E
P0	1	0	1	1	0	0	1	0	0	1	3	2	1	2	1
P1	1	1	0	0	0	0	0	1	0	1					
P2	0	0	0	1	0	0	0	0	0	1					
P3	0	0	0	0	0	1	0	1	0	1					

i. Convert this matrix representation to a resource allocation graph. 2 Marks

ii. Use the deadlock detection algorithm to determine whether the system contains a deadlock. 3 Marks

B. Consider the following snapshot of a system in which four resources A, B, C and D are available. The system contains a total of 2 instances of resource A, 2 instances of resource B, 1 instance of resource C and 1 instance of resource D.

	Allocation				**Maximum**				**Available**			
Process	A	B	C	D	A	B	C	D	A	B	C	D
P1	1	0	0	0	3	3	1	1	2	2	1	1
P2	0	1	0	0	1	1	0	2				
P3	1	0	1	0	3	2	1	0				
P4	0	0	0	1	3	1	1	1				

If a request from P3 arrives for (2, 2, 1, 0), can the request be granted immediately? Perform a safety test using Banker's Algorithm to determine the execution order and check for deadlocks.

5 Marks

Question 5 **10 Marks**

The following processes are being scheduled using a pre-emptive, round robin scheduling algorithm.

Process	**Priority**	**Burst Time (ms)**	**Arrival Time (ms)**
P1	40	20	0
P2	30	25	25
P3	30	25	30
P4	35	15	60
P5	5	10	100
P6	10	10	105

Each process is assigned a priority, with a higher number indicating a higher priority. The system also has an idle task (which consumes no CPU resources and memory and is identified as P-idle). This task has priority 0 and is scheduled whenever the system has no other available processes to run. The length of the time quantum is 10 units. If a process is pre-empted by a higher priority process, the pre-empted process is placed at the end of the queue.

i. Illustrate the Gantt chart showing the process schedule using pre-emptive round robin. 3 Marks

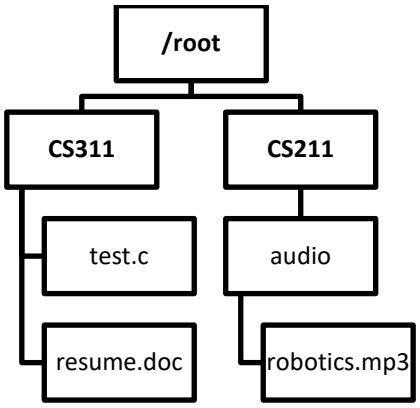
ii. Compute the turnaround and waiting times for each process. 2 Marks

iii. Compute the average waiting time and CPU utilisation rate? 2 Marks

iv. Discuss whether the scheduling algorithm effectively prioritized higher-priority processes and if there is any evidence of starvation among the processes. Would the shortest job first change the outcome of CPU utilisation. Justify your answers. 3 Marks

Question 6 **10 Marks**

A. The diagram below is a snapshot of directories and files in an Ubuntu operating system. Using the knowledge of **Linux Commands**, configure the following keeping in mind that the current directory is the root directory **/root**



i. Write the Linux command to create a new subdirectory called "Robotics" within CS311 directory. 1 Mark

ii. Write the Linux command to display the calendar of the year 2030 to a file called **calendar.txt** within CS311 directory. 1 Mark

iii. Write the Linux command to Copy the file robotics.mp3 to the CS311 directory. 1 Mark

iv. Rename the file resume.doc to cv.doc 1 Mark

v. Write the command to change directory to CS311 and then compile the file test.c 1 Mark

B. Effective memory management is essential for operating system performance. Consider a system with 100 available frames and the following processes:

- Process 1 (25KB), Process 2 (100KB), Process 3 (75KB)
- Process 4 (80KB), Process 5(115KB)

i. Compute the number of frames using a fixed allocation strategy. Justify whether all processes are significantly given enough frames. 1 Mark

ii. Calculate the number of frames using the proportional allocation strategy. Justify whether this will be relevant to a real-world scenario. 2 Marks

iii. Using a buddy system and 1MB of chunk memory available, how would the kernel allocate process 1 of 25KB. 2 Marks

Section C: Case Study 20 Marks

Consider a simple bank account with an initial balance. Multiple threads will simulate deposit and withdrawal transactions on this shared account concurrently.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/wait.h>

double account_balance = 1000.0;
pthread_mutex_t balance_mutex;

void *transaction(void *arg) {    char *type = (char *)arg;
    pthread_mutex_lock(&balance_mutex);
    if (strcmp(type, "deposit") == 0) {
        account_balance += 100.0;
        printf("Deposited. Bal: %.2f (P:%d,T:%lu)\n", account_balance, getpid(),
pthread_self());
    } else if (strcmp(type, "withdraw") == 0) {
        if (account_balance >= 200.0) {
            account_balance -= 200.0;
            printf("Withdrew. Bal: %.2f (P:%d,T:%lu)\n", account_balance, getpid(),
pthread_self());
        } else {
            printf("Insufficient funds (P:%d,T:%lu)\n", getpid(), pthread_self());
        }
    }
    pthread_mutex_unlock(&balance_mutex);
    return NULL;
}

int main() {
    pthread_t threads[4]; // Increased to 4 threads
    char *types[4] = {"deposit", "withdraw", "deposit", "withdraw"};
    int i;
    pid_t child_pid;

    pthread_mutex_init(&balance_mutex, NULL);
    child_pid = fork();

    if (child_pid == 0) {
        // Child: Only withdrawals
        pthread_t child_threads[2];
        char *child_types[2] = {"withdraw", "withdraw"};
        for (i = 0; i < 2; i++) {
            pthread_create(&child_threads[i], NULL, transaction, (void *)child_types[i]);
        }
        for (i = 0; i < 2; i++) {
            pthread_join(child_threads[i], NULL);
        }
        pthread_mutex_destroy(&balance_mutex);
        exit(0);
}
```

```
    else if (child_pid > 0) {
        // Parent: Both deposits and withdrawals
        for (i = 0; i < 4; i++) { // Loop 4 times for parent
            pthread_create(&threads[i], NULL, transaction, (void *)types[i]);
        }
        for (i = 0; i < 4; i++) {
            pthread_join(threads[i], NULL);
        }
        wait(NULL);
        pthread_mutex_destroy(&balance_mutex);
    } else {
        perror("fork");
        return 1;
    }
    return 0;
}
```

A. Explain the purpose of fork() system call in this program. Specifically, how many processes are created, and what is the relationship between them? 5 Marks

B. Describe the role of *pthread_mutex_lock()* and *pthread_mutex_unlock()* in the transaction function , explaining how they manage the sequence of operations when multiple threads interact with the account_balance variable. What potential problems could occur with the final account_balance value if these operations were not coordinated? 5 Marks

C. Analyse the memory space and threads created in the parent and child process and provide the values stored in each. 5 Marks

D. Explain why the parent process calls wait(Null). Is this necessary or would it not impact the outcome of any processes. If a parent process has terminated before the child process, what happens to the child process.
Include the concept of zombie processes and orphan processes in your discussion.
5 Marks